

Competitive Exam Practice 6 — Answer Key

Singly Linked Lists

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 26, 2026

- Q1 (B) Binary search.** Binary search needs $O(1)$ random access to the middle element, but reaching the middle of a linked list costs a linear walk — so the halving step is $O(n)$, not $O(1)$, and the $O(\log n)$ bill collapses. Insertion sort (splice without shifting), radix sort (sequential bucket appends), and polynomial manipulation (one node per nonzero term) all suit lists. Confirmed on the GATEOverflow page, solutionsadda's GATE-1994 listing, the edurev GATE CS 1994 paper, and ExamSIDE's GATE-1994 listing (2026-09-26).
- Q2 (D) n .** A singly linked list has no indexing, so only linear search applies: the miss (or hit at the last node) compares against all n elements. $n/2$ is the *average*-case figure, and the log options wrongly assume binary search, which needs $O(1)$ mid-access. Confirmed on the GATEOverflow page (answer D), the gatesolutions blog (answer (d) n), the edurev GATE CS 2002 paper (Q1.5), and ExamSIDE's GATE-2002 listing (2026-09-26).
- Q3 (B) The elements are sorted in non-decreasing order.** The recursion demands `p->data <= p->next->data` at *every* adjacent pair, which is exactly non-decreasing order (equal neighbours allowed). (A) is contained in (B) but not equivalent — `f` also returns 1 on longer sorted lists, so “if and only if” rules (A) out; (C) flips the comparison; (D) is neither necessary nor sufficient. Confirmed on the GATEOverflow page (answer B), ExamSIDE's GATE-2003 listing, electrical4u's GATE CS 2003 solutions (answer B), and Knowledge Gate's SLL PYQ page (2026-09-26).
- Q4 (B) 2, 1, 4, 3, 6, 5, 7.** Each pass swaps one adjacent pair by *value* and then skips both nodes (`p = q->next`), so pairs (1, 2), (3, 4), (5, 6) are swapped and the odd node 7 has no partner and stays put. (A) would need zero swaps; (C) shifts the pairing by one; (D) rotates instead of swapping. Confirmed on the GATEOverflow page (answer B), ExamSIDE's GATE-2008 listing, electrical4u's GATE CS 2008 solutions (answer B), and the EduRev discussion (2026-09-26).
- Q5 (D) `q->next = NULL; p->next = head; head = p;`** On exit `p` is the last node and `q` its predecessor: detach the old tail link first (`q->next = NULL`), point the last node at the old head, then swing `head` over — in exactly this order. (A) orphans the whole rest of the list (`q` is dropped, tail never detached); (B) sets `head = p` before linking so `p->next = head` is a self-loop; (C) mislinks `p->next` to `q`. Confirmed on the GATEOverflow page (answer D), the GFG PYQ quiz, compsciedu's MCQ page (answer (d)), electrical4u's GATE CS 2010 solutions (answer D), and ExamSIDE's GATE-2010 listing (2026-09-26).

- Q6 (A) The best algorithm takes $\Theta(n)$ time in the worst case.** The deck's three-pointer loop visits each node exactly once and flips one link per node — n flips, $\Theta(n)$ worst case, with only three extra pointers ($O(1)$ space). No $o(n)$ algorithm can touch all n links, ruling out (B)/(C); (D) is refuted by the very loop above. Confirmed on the GATEOverflow page (answer A), the official Collegedunia GATE-2022 paper+solutions PDF (correct answer A), Garg's Academy solutions (answer A), ExamSIDE's GATE-2022 listing, and GFG's quiz page (2026-09-26).
- Q7 (A) `newNode->next = head; then head = newNode;` giving `head` $\rightarrow 4 \rightarrow 11 \rightarrow 23 \rightarrow \text{NULL}$.** The new box must point at the old chain *before* `head` moves; reversing the order ((B)/(D)) overwrites `head`'s old value with no copy anywhere and orphans `11` $\rightarrow$ `23`. (C) keeps the safe order but misstates the result (front-insert never appends). Confirmed by execution (2026-09-26).
- Q8 (A) The list is `head` $\rightarrow 8 \rightarrow 29 \rightarrow \text{NULL}$; the bypass is $O(1)$ worst case once the predecessor is in hand, and the overall delete is $O(n)$ worst case because of the search.** Node 8's `next` is rewritten to skip 13 straight to 29, then 13's box is freed. (B) leaves the list unchanged; (C) deletes the wrong node and claims the search is free; (D) gets the list right but drops the $O(n)$ search bill the deck's price list charges. Confirmed by execution (2026-09-26).